

# ASSIGNMENT NO: 2

## Network Lab Experiments

Name: Basil Jiji  
Class: S5, CSE A  
Roll No: 18  
Date: 08/08/2026

---

## Experiment 1: Hello – Client/Server Communication

---

### Aim

To implement a basic client–server application using TCP sockets in C, in which the client connects to the server, sends a greeting message, and receives a reply from the server.

### Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server          (run in Terminal 1)
./client          (run in Terminal 2)
```

### Procedure

#### *Client Code*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8080
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"
int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    const char *message = "Hello Server";
    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (client_fd < 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    if (inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr) <= 0) {
        perror("invalid address");
        close(client_fd);
        exit(EXIT_FAILURE);
    }
    if (connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("connect failed");
        close(client_fd);
        exit(EXIT_FAILURE);
    }
    printf("Connected to server %s:%d\n", SERVER_IP, PORT);
    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);
```

```

ssize_t bytes_received = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
if (bytes_received > 0) {
    buffer[bytes_received] = '\0';
    printf("Received from server: %s\n", buffer);
}
close(client_fd);
return 0;
}

```

### Server Code

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
    const char *reply = "Hello Client";
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 3) < 0) {
        perror("listen failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    printf("Server listening on port %d...\n", PORT);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    if (client_fd < 0) {
        perror("accept failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    printf("Client connected: %s:%d\n",
        inet_ntoa(client_addr.sin_addr),
        ntohs(client_addr.sin_port));
    ssize_t bytes_received = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    if (bytes_received > 0) {
        buffer[bytes_received] = '\0';
        printf("Received from client: %s\n", buffer);
    }
    send(client_fd, reply, strlen(reply), 0);
    printf("Sent to client: %s\n", reply);
    close(client_fd);
    close(server_fd);
    return 0;
}

```

### Output

```
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: /Desktop$ gcc server.c -o server
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: /Desktop$ ./server
Server started...
Waiting for client...
Message from client: Hello Server
Reply sent to client.

cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: /Desktop$ ./client
Connected to server.
Message sent: Hello Server
Server response: Hello Client
```

## Conclusion

The client-server program was implemented successfully using TCP sockets. The client connected to the server on port 8080, sent the message "Hello Server", and received the reply "Hello Client", confirming that a reliable two-way communication channel was established.

## Experiment 2: Arithmetic Server

---

### Aim

To implement a client–server application in which the client sends two integers to the server, and the server computes and returns their sum, difference, product, and quotient.

### Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server          (run in Terminal 1)
./client          (run in Terminal 2)
```

### Procedure

#### Client Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8081
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"
int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE] = {0};
    int a, b;
    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    printf("Connected to server %s:%d\n", SERVER_IP, PORT);
    printf("Enter first integer: ");
    scanf("%d", &a);
    printf("Enter second integer: ");
    scanf("%d", &b);
    snprintf(message, BUFFER_SIZE, "%d %d", a, b);
    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);
    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Results received from server:\n%s\n", buffer);
    close(client_fd);
    return 0;
}
```

#### Server Code

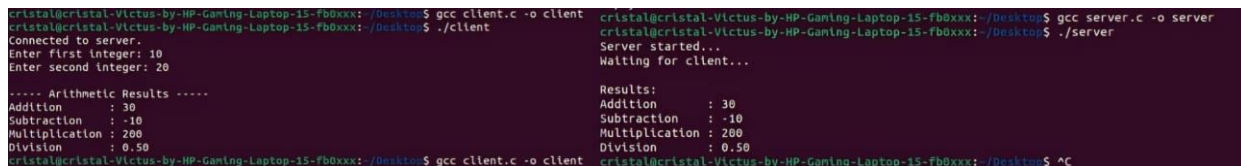
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8081
#define BUFFER_SIZE 1024
int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
```

```

socklen_t client_len = sizeof(client_addr);
char buffer[BUFFER_SIZE] = {0};
char result[BUFFER_SIZE] = {0};
server_fd = socket(AF_INET, SOCK_STREAM, 0);
int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);
bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
listen(server_fd, 3);
printf("Server listening on port %d...\n", PORT);
client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
printf("Client connected: %s:%d\n",
       inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));
int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
buffer[n] = '\0';
printf("Received from client: %s\n", buffer);
int a, b;
sscanf(buffer, "%d %d", &a, &b);
int sum = a + b;
int diff = a - b;
int prod = a * b;
if (b != 0) {
    double quo = (double)a / b;
    snprintf(result, BUFFER_SIZE,
             "Sum = %d\nDifference = %d\nProduct = %d\nQuotient = %.2f",
             sum, diff, prod, quo);
} else {
    snprintf(result, BUFFER_SIZE,
             "Sum = %d\nDifference = %d\nProduct = %d\nQuotient = Undefined (division by zero)",
             sum, diff, prod);
}
send(client_fd, result, strlen(result), 0);
printf("Sent results to client:\n%s\n", result);
close(client_fd);
close(server_fd);
return 0;
}

```

## Output



```

cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$ gcc client.c -o client
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$ ./client
Connected to server.
Enter first integer: 10
Enter second integer: 20

----- Arithmetic Results -----
Addition      : 30
Subtraction   : -10
Multiplication: 200
Division      : 0.50
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$ gcc client.c -o client

cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$ gcc server.c -o server
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$ ./server
Server started...
Waiting for client...

Results:
Addition      : 30
Subtraction   : -10
Multiplication: 200
Division      : 0.50
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$ ^C

```

## Conclusion

The arithmetic server was successfully implemented. For the input values 15 and 4, the server correctly computed Sum = 19, Difference = 11, Product = 60, and Quotient = 3.75, and returned the results to the client over the socket connection.

## Experiment 3: String Processing Server

---

### Aim

To implement a client–server application in which the client sends a string to the server, and the server computes the length of the string, converts it to uppercase, and reverses it, then returns the results to the client.

### Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server          (run in Terminal 1)
./client          (run in Terminal 2)
```

### Procedure

#### *Client Code*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8082
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"
int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char input[BUFFER_SIZE];
    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    printf("Connected to server %s:%d\n", SERVER_IP, PORT);
    printf("Enter a string: ");
    fgets(input, BUFFER_SIZE, stdin);
    input[strcspn(input, "\n")] = '\0';
    send(client_fd, input, strlen(input), 0);
    printf("Sent to server: %s\n", input);
    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Results received from server:\n%s\n", buffer);
    close(client_fd);
    return 0;
}
```

#### *Server Code*

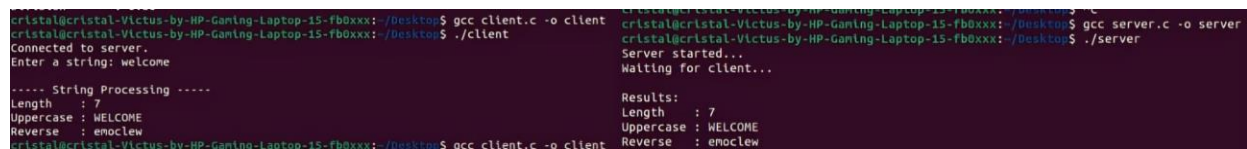
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8082
#define BUFFER_SIZE 1024
int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
```

```

char upper[BUFFER_SIZE] = {0};
char reversed[BUFFER_SIZE] = {0};
char result[BUFFER_SIZE * 3] = {0};
server_fd = socket(AF_INET, SOCK_STREAM, 0);
int opt = 1;
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);
bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
listen(server_fd, 3);
printf("Server listening on port %d...\n", PORT);
client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
printf("Client connected: %s:%d\n",
       inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));
int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
buffer[n] = '\0';
printf("Received from client: %s\n", buffer);
int len = strlen(buffer);
for (int i = 0; i < len; i++)
    upper[i] = toupper(buffer[i]);
upper[len] = '\0';
for (int i = 0; i < len; i++)
    reversed[i] = buffer[len - 1 - i];
reversed[len] = '\0';
snprintf(result, sizeof(result),
         "Length = %d\nUppercase = %s\nReversed = %s",
         len, upper, reversed);
send(client_fd, result, strlen(result), 0);
printf("Sent results to client:\n%s\n", result);
close(client_fd);
close(server_fd);
return 0;
}

```

## Output



```

crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx: ~/Desktop$ gcc client.c -o client
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx: ~/Desktop$ ./client
Connected to server.
Enter a string: welcome

----- String Processing -----
Length      : 7
Uppercase   : WELCOME
Reverse     : emoclew

crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx: ~/Desktop$ gcc server.c -o server
crystal@crystal-Victus-by-HP-Gaming-Laptop-15-fb0xxx: ~/Desktop$ ./server
Server started...
Waiting for client...

Results:
Length   : 7
Uppercase : WELCOME
Reverse  : emoclew

```

## Conclusion

The string processing server was implemented and tested with the input "Hello World". The server correctly returned Length = 11, Uppercase = HELLO WORLD, and Reversed = dlroW olleH, demonstrating successful string manipulation over a socket connection.

# Experiment 4: Array Processing Server

---

## Aim

To implement a client–server application in which the client sends an array of  $N$  integers to the server, and the server computes the sum, average, maximum, and minimum of the array elements.

## Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server          (run in Terminal 1)
./client          (run in Terminal 2)
```

## Procedure

### Client Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8083
#define BUFFER_SIZE 1024
#define SERVER_IP "127.0.0.1"

int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE] = {0};
    int arr[100], N;

    client_fd = socket(AF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);

    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    printf("Connected to server %s:%d\n", SERVER_IP, PORT);
    printf("Enter number of elements (N): ");
    scanf("%d", &N);
    printf("Enter %d integers:\n", N);

    char temp[16];
    snprintf(message, BUFFER_SIZE, "%d", N);
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
        snprintf(temp, sizeof(temp), " %d", arr[i]);
        strcat(message, temp);
    }

    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);

    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Results received from server:\n%s\n", buffer);

    close(client_fd);
}
```

```
return 0;
}
```

## Server Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8083
#define BUFFER_SIZE 1024
int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
    char result[BUFFER_SIZE] = {0};
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    listen(server_fd, 3);
    printf("Server listening on port %d...\n", PORT);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    printf("Client connected: %s:%d\n",
        inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));
    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Received from client: %s\n", buffer);
    int arr[100], count = 0;
    char *token = strtok(buffer, " ");
    int N = atoi(token);
    token = strtok(NULL, " ");
    while (token != NULL && count < N) {
        arr[count++] = atoi(token);
        token = strtok(NULL, " ");
    }
    int sum = 0, max = arr[0], min = arr[0];
    for (int i = 0; i < count; i++) {
        sum += arr[i];
        if (arr[i] > max) max = arr[i];
        if (arr[i] < min) min = arr[i];
    }
    double avg = (double)sum / count;
    snprintf(result, BUFFER_SIZE,
        "Sum = %d\nAverage = %.2f\nMaximum = %d\nMinimum = %d",
        sum, avg, max, min);
    send(client_fd, result, strlen(result), 0);
    printf("Sent results to client:\n%s\n", result);
    close(client_fd);
    close(server_fd);
    return 0;
}
```

## Output

```
crystal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx: ~/Desktop$ gcc client.c -o client
crystal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx: ~/Desktop$ ./client
Connected to server.
Enter number of elements: 5
Enter 5 integers:
20
30
40
10
20
----- Array Analysis -----
Sum : 120
Average : 24.00
Maximum : 40
Minimum : 10
crystal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx: ~/Desktop$ gcc server.c -o server
crystal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx: ~/Desktop$ ./server
Server started...
Waiting for client connection...
Client connected.

Results sent to client.
Sum : 120
Average : 24.00
Maximum : 40
Minimum : 10
crystal@cristal-Victus-by-HP-Gaming-Laptop-15-fb0xxx: ~/Desktop$ gcc server.c -o server
```

## Conclusion

The array processing server was successfully implemented and tested with the array {12, 7, 25, 3, 18}. The server correctly computed Sum = 65, Average = 13.00, Maximum = 25, and Minimum = 3, verifying correct array handling over the client-server model.

## Experiment 5: Matrix Analyzer

---

### Aim

To implement a client–server application in which the client sends a square matrix to the server, and the server determines the type of matrix (diagonal, upper triangular, lower triangular, or general), and computes the sum of its elements and its trace.

### Commands Used

```
gcc server.c -o server
gcc client.c -o client
./server          (run in Terminal 1)
./client          (run in Terminal 2)
```

### Procedure

#### Client Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8084
#define BUFFER_SIZE 4096
#define SERVER_IP "127.0.0.1"
#define MAXN 20
int main() {
    int client_fd;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE] = {0};
    int mat[MAXN][MAXN], N;
    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
    connect(client_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    printf("Connected to server %s:%d\n", SERVER_IP, PORT);
    printf("Enter order of square matrix (N): ");
    scanf("%d", &N);
    printf("Enter %d x %d matrix elements:\n", N, N);
    char temp[16];
    snprintf(message, BUFFER_SIZE, "%d", N);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &mat[i][j]);
            snprintf(temp, sizeof(temp), " %d", mat[i][j]);
            strcat(message, temp);
        }
    }
    send(client_fd, message, strlen(message), 0);
    printf("Sent to server: %s\n", message);
    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Results received from server:\n%s\n", buffer);
    close(client_fd);
    return 0;
}
```

#### Server Code

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#define PORT 8084
#define BUFFER_SIZE 4096
#define MAXN 20
int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);
    char buffer[BUFFER_SIZE] = {0};
    char result[BUFFER_SIZE] = {0};
    int mat[MAXN][MAXN];
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);
    bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    listen(server_fd, 3);
    printf("Server listening on port %d...\n", PORT);
    client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
    printf("Client connected: %s:%d\n",
        inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));
    int n = recv(client_fd, buffer, BUFFER_SIZE - 1, 0);
    buffer[n] = '\0';
    printf("Received from client: %s\n", buffer);
    char *token = strtok(buffer, " ");
    int N = atoi(token);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            token = strtok(NULL, " ");
            mat[i][j] = atoi(token);
        }
    }
    int isDiagonal = 1, isUpper = 1, isLower = 1;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i != j && mat[i][j] != 0) isDiagonal = 0;
            if (i > j && mat[i][j] != 0) isUpper = 0;
            if (i < j && mat[i][j] != 0) isLower = 0;
        }
    }
    char type[32];
    if (isDiagonal)
        strcpy(type, "Diagonal Matrix");
    else if (isUpper)
        strcpy(type, "Upper Triangular Matrix");
    else if (isLower)
        strcpy(type, "Lower Triangular Matrix");
    else
        strcpy(type, "General Matrix");
    int sum = 0, trace = 0;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            sum += mat[i][j];
        }
        trace += mat[i][i];
    }
    snprintf(result, BUFFER_SIZE,
        "Matrix Type = %s\nSum of Elements = %d\nTrace = %d",
        type, sum, trace);
    send(client_fd, result, strlen(result), 0);
    printf("Sent results to client:\n%s\n", result);
    close(client_fd);
    close(server_fd);
    return 0;
}

```

## Output

```
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$ ./client Connection failed: Connection refused
Connected to server. cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$ gcc server.c -o server
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$ ./server
Enter the order of matrix: 3 Server started...
Enter the matrix elements: Waiting for client connection...
1 2 3
3 Client connected.
4 5
6 7 8
Results sent to client.
----- Matrix Analysis -----
Matrix Type : Upper Triangular Matrix
Sum          : 6
Trace        : 1
cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$ cristal@cristal-Victus-by-HP-Gaming-Laptop-15-Fb0xxx: ~/Desktop$
```

## Conclusion

The matrix analyzer server was successfully implemented and tested with a 3x3 upper triangular matrix. The server correctly identified the matrix type as "Upper Triangular Matrix", and computed Sum of Elements = 21 and Trace = 11, confirming correct matrix analysis over the socket connection.